CIT CAMPUS PLATFORM

Source Data Audit & Data Dictionary

Technical foundation · Version 0.1 · Working draft

Dataset: cit-public-v1
Source: CrunchWeek Starter Pack
Institution: Chilika Institute of Technology (fictional)

Prepared as the implementation source of truth for database, backend, scheduling, booking, exams, disruptions, Campus Navigator and AI integration.

Authority rule

The official starter-pack JSON and validator.py define the competition input/output contract. This document separates source facts from proposed application architecture. Where the validator does not enforce a business rule, this document explicitly identifies that gap.

1. Executive summary

The starter pack provides one canonical JSON dataset, supporting CSV extracts, an official Python validator, a sample disruption, sample natural-language booking requests and an example schedule export. Judges may reset the application, generate a timetable and apply their own change files. Required exports must be produced by the application, not edited by hand.

2. Verified source inventory

Collection	Count	Meaning
rooms	27	Rooms
room_unavailability	2	Room unavailability entries
faculty	40	Faculty
student_groups	14	Student groups
courses	62	Courses
sessions	189	Schedulable sessions
booking_requests	25	Spectra booking requests
cancellations	1	Cancellations
exams	41	Midsem exams

189 sessions represent 243 teaching hours per week. The public dataset contains one cancelled booking. The README states that a zero-hard-violation timetable exists. JSON is the complete canonical source; CSVs are supporting views and omit root-level configuration.

3. Canonical source and import rules

• `data/cit_public.json` is authoritative; CSVs must not be the only import source.

• Preserve the original source file and checksum.

• Parse and validate the complete JSON before writing records.

• Import transactionally; a dataset version is either fully imported or not imported.

• Preserve external IDs, nulls, empty arrays, timestamps and raw request text.

• Do not silently coerce invalid references.

• Record import provenance, counts, warnings, rejected records and unresolved references.

4. Full source data dictionary

Root.dataset_id

Type: string **Presence:** Required
Meaning: Dataset identity/version token. Current value cit-public-v1.
Implementation: Unique dataset identifier; preserve exactly in exports.

Root.institute

Type: string **Presence:** Required
Meaning: Institute display name; source marks institution fictional.
Implementation: Configuration metadata.

Root.calendar.days

Type: array **Presence:** Required
Meaning: Allowed teaching weekdays, MON–FRI.
Implementation: Do not hardcode weekdays; read from imported calendar.

Root.calendar.periods[].id

Type: integer **Presence:** Required
Meaning: Period identifier 1–13.
Implementation: Unique within calendar; periods 1–9 teaching, 10–13 event-only in this dataset.

Root.calendar.periods[].start/end

Type: HH:MM string **Presence:** Required
Meaning: Local clock boundaries for each period.
Implementation: Store as time or normalized minute offsets; preserve source values for export.

Root.calendar.periods[].teaching

Type: boolean **Presence:** Required
Meaning: Whether period is a teaching period.
Implementation: Solver must use this flag, not infer solely from period number.

Root.calendar.lunch.after_period

Type: integer **Presence:** Required
Meaning: Lunch follows the specified period; current value 5.
Implementation: Validator rejects teaching sessions crossing P5/P6.

Root.calendar.lunch.start/end

Type: HH:MM string **Presence:** Required
Meaning: Lunch interval boundaries.
Implementation: Calendar configuration; not a session.

Root.calendar.lab_start_periods

Type: array **Presence:** Required
Meaning: Permitted lab start periods; current [2,6].
Implementation: Hard scheduling rule for kind=lab.

Root.calendar.event_only_periods

Type: array **Presence:** Required
Meaning: Periods available for events; current [10,11,12,13].
Implementation: Events can use evening periods; regular sessions cannot.

Root.calendar.fest_week

Type: string **Presence:** Required
Meaning: Human-readable fest-week label.
Implementation: Metadata; Week 7 Spectra '27 in current source.

Root.calendar.exam_week.days

Type: array **Presence:** Required
Meaning: Allowed exam days; current MON–SAT.
Implementation: Separate from teaching calendar days.

Root.calendar.exam_week.sessions[].id/start/end

Type: string / HH:MM **Presence:** Required
Meaning: Exam session identifier and clock window; AM and PM.
Implementation: Exam schedule slot is (day, session_id), not teaching period.

Root.priority_order

Type: array **Presence:** Required
Meaning: Ordered request categories, highest priority first.
Implementation: Do not hardcode; effective category determined using role cap.

Root.role_category_cap

Type: object **Presence:** Required
Meaning: Maximum category a role may claim.
Implementation: Role caps apply to requested category via effective_category().

rooms[].id

Type: string **Presence:** Required
Meaning: Stable room code, e.g. AUD, CR-101.
Implementation: Primary key; referenced by sessions, unavailability, booking allocations and exam allocations.

rooms[].name

Type: string **Presence:** Required
Meaning: Human-readable room name.
Implementation: Display field.

rooms[].building

Type: string **Presence:** Required
Meaning: Building name as provided by source.
Implementation: No separate building entity in starter data; normalize to buildings only if campus-map records are verified.

rooms[].type

Type: enum-like string **Presence:** Required
Meaning: Room category (classroom, lecture_hall, auditorium, seminar_hall, outdoor, lab_cs, lab_elec, lab_phy, workshop, lab_civil).
Implementation: Use source value; don't infer types from room name.

rooms[].capacity

Type: integer **Presence:** Required
Meaning: Normal room capacity.
Implementation: For exams, validator uses floor(capacity/2) per room.

rooms[].features

Type: array **Presence:** Required
Meaning: Room equipment/features, e.g. projector, stage, pa_system, ac.
Implementation: Normalize to room_feature join table or JSONB; retain exact source feature codes.

rooms[].accessible

Type: boolean **Presence:** Required
Meaning: Whether room is accessible.
Implementation: Booking feature matching treats accessible as a feature when true.

rooms[].allowed_uses

Type: array **Presence:** Required
Meaning: Allowed activity categories, e.g. lecture, tutorial, lab, exam, event.
Implementation: Hard gate for exam/event allocation; source README notes seminar halls and OAT do not host regular classes.

room_unavailability[].room_id

Type: string **Presence:** Required
Meaning: Room that is unavailable.
Implementation: FK to rooms.id.

room_unavailability[].day

Type: string **Presence:** Required
Meaning: Unavailable weekday.
Implementation: Validate against calendar days.

room_unavailability[].periods

Type: array **Presence:** Required
Meaning: Unavailable period identifiers.
Implementation: One record can represent multiple periods; recommended relational expansion to one row per period.

room_unavailability[].reason

Type: string **Presence:** Required
Meaning: Reason for unavailability.
Implementation: Preserve for UI and audit.

faculty[].id

Type: string **Presence:** Required
Meaning: Faculty identifier, e.g. F01.
Implementation: Primary key.

faculty[].name

Type: string **Presence:** Required
Meaning: Faculty display name.
Implementation: Personal data; restrict to authorised academic users.

faculty[].department

Type: string **Presence:** Required
Meaning: Department code.
Implementation: Department reference; source does not provide a separate department master table.

faculty[].max_hours_per_day

Type: integer **Presence:** Required in source; validator fallback 4
Meaning: Hard daily teaching limit.
Implementation: Persist explicitly; do not silently replace missing values with fallback.

faculty[].needs_accessible

Type: boolean **Presence:** Required
Meaning: Faculty accessibility requirement.
Implementation: Room accessibility is checked for sessions only when session.needs_accessible is true; faculty flag is source metadata and should be mapped carefully. Do not assume validator enforces it.

faculty[].unavailable[].day/periods/reason

Type: object **Presence:** Optional/repeatable
Meaning: Faculty time unavailability with reason.
Implementation: Expand periods into individual normalized rows; enforce as hard constraint.

faculty[].preferences[].type/days/periods/note

Type: object **Presence:** Optional/repeatable
Meaning: Soft preference; documented avoid type.
Implementation: Current sample may be empty for some/all faculty. For avoid preference, penalty is 2 per overlapping hour.

student_groups[].id

Type: string **Presence:** Required
Meaning: Student group identifier, e.g. Y1-A.
Implementation: Primary key; referenced by sessions and exams.

student_groups[].name

Type: string **Presence:** Required
Meaning: Display name.
Implementation: Display field.

student_groups[].year

Type: integer **Presence:** Required
Meaning: Academic year.
Implementation: Filter/grouping field.

student_groups[].department

Type: string **Presence:** Required
Meaning: Department code; COMMON appears for first-year shared groups.
Implementation: Department reference or code.

student_groups[].size

Type: integer **Presence:** Required
Meaning: Nominal group size.
Implementation: Source metadata; session.size and exam.size are the attendance/capacity values used by validator.

student_groups[].needs_accessible

Type: boolean **Presence:** Required
Meaning: Group accessibility requirement.
Implementation: Validator uses session.needs_accessible and booking needs; it does not directly inspect group flag in check_timetable.

courses[].id

Type: string **Presence:** Required
Meaning: Course identifier.
Implementation: Primary key; referenced by sessions and exams.

courses[].title

Type: string **Presence:** Required
Meaning: Course title.
Implementation: Display field.

courses[].department

Type: string **Presence:** Required
Meaning: Owning department code.
Implementation: Academic filtering.

courses[].year

Type: integer **Presence:** Required
Meaning: Course year.
Implementation: Academic filtering.

courses[].lecture_hours/tutorial_hours/lab_blocks

Type: integer **Presence:** Required
Meaning: Reference workload counts.
Implementation: These are reference data; solver schedules sessions, not course-hour totals directly.

sessions[].id

Type: string **Presence:** Required
Meaning: Unique schedulable teaching unit.
Implementation: Primary key; exactly one assignment required.

sessions[].course_id

Type: string **Presence:** Required
Meaning: Owning course.
Implementation: FK to courses.id.

sessions[].kind

Type: enum-like string **Presence:** Required
Meaning: lecture, tutorial or lab.
Implementation: Lab start restriction applies when kind=lab.

sessions[].groups

Type: array **Presence:** Required
Meaning: Every student group that must attend.
Implementation: Many-to-many relationship; use session_groups join table.

sessions[].faculty_id

Type: string **Presence:** Required
Meaning: Assigned faculty member.
Implementation: FK to faculty.id; source preassigns faculty.

sessions[].size

Type: integer **Presence:** Required
Meaning: Number of attendees for this session.
Implementation: Used for room capacity; not necessarily sum of group sizes.

sessions[].duration

Type: integer **Presence:** Required
Meaning: Number of consecutive periods occupied.
Implementation: One hour per period in teaching timetable.

sessions[].room_types

Type: array **Presence:** Required
Meaning: Allowed room types.
Implementation: Session can use any room whose type is in this array.

sessions[].needs_accessible

Type: boolean **Presence:** Required
Meaning: Whether assigned room must be accessible.
Implementation: Hard constraint H10.

sessions[].basket_slot

Type: string|null **Presence:** Nullable
Meaning: Elective basket identifier.
Implementation: Sessions with same basket_slot must have identical day and start period; basket sessions in same slot are treated as one group-occupancy key.

sessions[].fixed_slot

Type: object|null **Presence:** Nullable
Meaning: Fixed day and start_period.
Implementation: When present, exact day and start period must be preserved (H14).

booking_requests[].id

Type: string **Presence:** Required
Meaning: Booking request identifier.
Implementation: Primary key.

booking_requests[].submitted_at

Type: ISO-8601 datetime with offset **Presence:** Required
Meaning: Submission timestamp.
Implementation: Tie-breaker for same effective priority; earlier timestamp wins.

booking_requests[].requester

Type: string **Presence:** Required
Meaning: Requester display/entity name.
Implementation: Source data does not include an authenticated user ID; app must link imported requester to account where possible without inventing identity.

booking_requests[].role

Type: enum-like string **Presence:** Required
Meaning: admin, department_coordinator, faculty, club.
Implementation: Used with role_category_cap.

booking_requests[].department

Type: string|null **Presence:** Nullable
Meaning: Department context for request.
Implementation: No department FK is guaranteed by source; validate against configured department list if introduced.

booking_requests[].requested_category

Type: string **Presence:** Required
Meaning: Requested event category.
Implementation: Effective category is adjusted by role cap.

booking_requests[].title

Type: string **Presence:** Required
Meaning: Booking/event title.
Implementation: Display field.

booking_requests[].attendance

Type: integer **Presence:** Required
Meaning: Expected attendance.
Implementation: Room capacity requirement.

booking_requests[].duration_periods

Type: integer **Presence:** Required
Meaning: Requested consecutive period count.
Implementation: Allocated inclusive range must have exactly this length.

booking_requests[].windows[].day/from_period/to_period

Type: object **Presence:** Required/repeatable
Meaning: Allowed requested windows; end period inclusive.
Implementation: Approved bookings must fit entirely inside one window; alternative_offered may be outside.

booking_requests[].room_types

Type: array **Presence:** Required
Meaning: Acceptable room types.
Implementation: Candidate room filter.

booking_requests[].needs

Type: array **Presence:** Required
Meaning: Required room features; accessible is synthesized from room.accessible.
Implementation: All requested needs must be satisfied.

booking_requests[].requested_room

Type: string|null **Presence:** Nullable
Meaning: Preferred room, may be null or unknown.
Implementation: Preference only; validator does not require this exact room.

booking_requests[].mandatory_for_groups

Type: array **Presence:** Required
Meaning: Groups that must be free during booking.
Implementation: Conflict check against Week 7 timetable.

booking_requests[].series_id

Type: string|null **Presence:** Nullable
Meaning: Recurring booking series reference.
Implementation: Starter schema only; recurring allocation semantics not specified by validator.

booking_requests[].raw_text

Type: string **Presence:** Required
Meaning: Original natural-language request.
Implementation: Keep immutable for audit, parsing evaluation and AI provenance.

cancellations[].request_id

Type: string **Presence:** Required
Meaning: Cancelled booking request.
Implementation: FK to booking_requests.id; cancellation blocks approval/alternative.

cancellations[].cancelled_at

Type: ISO-8601 datetime with offset **Presence:** Required
Meaning: Cancellation timestamp.
Implementation: Audit field.

cancellations[].by

Type: string **Presence:** Required
Meaning: Cancelling entity label.
Implementation: Source doesn't provide account ID.

cancellations[].note

Type: string **Presence:** Required
Meaning: Cancellation reason/note.
Implementation: Preserve.

exams[].id

Type: string **Presence:** Required
Meaning: Exam identifier.
Implementation: Primary key.

exams[].course_ids

Type: array **Presence:** Required
Meaning: Courses covered by exam.
Implementation: Many-to-many to courses.

exams[].groups

Type: array **Presence:** Required
Meaning: Groups sitting exam.
Implementation: Many-to-many to student groups.

exams[].size

Type: integer **Presence:** Required
Meaning: Total exam attendance.
Implementation: Validator compares to sum of floor(room.capacity/2).

exams[].duration_hours

Type: integer **Presence:** Required
Meaning: Exam duration.
Implementation: Exam session windows currently two hours; validator does not explicitly compare duration_hours to selected AM/PM duration.

5. Calendar and time semantics

• Teaching days are MON–FRI. P1–P9 are teaching periods; P10–P13 are event-only.

• Lunch is 13:00–14:00 after P5. A session cannot span P5/P6.

• Labs start only at P2 or P6.

• Exam week is MON–SAT, with AM 09:30–11:30 and PM 14:00–16:00.

• Exam room effective capacity is floor(normal room capacity / 2).

• Teaching periods, event periods and exam sessions are separate scheduling domains.

6. Official validator contract

Hard timetable constraints

Rule	Validator behaviour
H1	Every source session exactly once; unknown session/room invalid; duplicate output IDs reported.
H2	No faculty overlap.
H3	No student-group overlap; same basket key is treated as one occupancy key.
H4	No room overlap.
H5	Room capacity ≥ session size.
H6	Room type must be in session.room_types.

Rule	Validator behaviour
H7/H8	Faculty and room unavailability respected.
H9	P1–P9 only, no lunch crossing, lab starts at configured lab periods.
H10	Session requiring accessibility uses accessible room.
H11	Same basket_slot sessions have identical day and start period.
H12	Faculty daily duration ≤ max_hours_per_day (fallback 4 if missing).
H14	fixed_slot day and start_period preserved.

Soft timetable penalties

Rule	Penalty
S1	2 per occupied period overlapping faculty avoid preference.
S2	1 per gap period between a group's first and last occupied period each day.
S3	3 per group teaching hour above 6 hours/day.
S4	3 per additional same-course lecture for a group/day; basket course repetitions scored separately.
S5	1 per session using less than 40% of room capacity.
S6	1 per session occupying P9.

Booking validation

• B1: duplicate decisions for one request are invalid.

• B2: overrides must reference a known session and an approved booking; only institute events may displace regular sessions.

• B3: cancelled requests cannot be approved or offered alternatives.

• Every request needs a decision; missing decisions are warnings.

• Allowed statuses: approved, rejected, alternative_offered. Rejections and alternatives need a reason.

• Room, event permission, type, capacity, features, time window, room unavailability, timetable occupancy and mandatory-group occupancy are checked.

• Approved bookings must fit fully within one requested window; alternative offers may be outside the window.

• Priority inversions are reported. Effective category applies role caps; same-category tie goes to earlier submitted_at.

• Validator weights: institute event 5, guest lecture 4, department event 3, club event 2.

The validator evaluates exported decisions; it does not implement authenticated reviewer assignment, no-self-approval, multi-stage approval or complete operational workflows. Those belong in backend logic.

Exam validation

• Every source exam must be scheduled exactly once.

• Day and exam session must be valid.

• Rooms must exist and allow exams; combined half-capacity must cover exam size.

• A room or student group cannot have overlapping exams in the same exam slot.

• A group has a soft penalty of 5 for each additional exam on the same day.

• The validator does not explicitly compare source duration_hours against AM/PM duration; preserve it without inventing a validator rule.

7. Disruption contract

Supported change types: faculty_unavailable, room_unavailable, add_faculty, add_session and remove_session. The validator applies changes to a deep copy of the dataset. Changes are permanent in the derived dataset for subsequent weeks. Retain the

original dataset, store immutable change sets, create a derived dataset version after approval, preserve the baseline timetable and compare the repaired schedule against it. Week 7 session overrides are not permanent disruptions.

Public sample disruption

F07 unavailable Monday P1–P9; CR-102 unavailable Monday–Friday P1–P9; add session CS202-T2 for course CS202, group CSE-2, faculty F06, size 80, duration 1.

8. Export contract

schedule.json requires team, dataset_id, sessions, fest_week and exams. Each session assignment contains session_id, day, start_period and room_id. fest_week contains bookings and session_overrides. Booking end_period is inclusive. Exam assignments contain exam_id, day, session and rooms[]. Only an approved institute event may displace a regular session; the displaced session must be moved, not dropped.

schedule_after.json is the repaired timetable after applying the disruption, starting from schedule.json. Generate both exports from persisted state, run the official validator, save the report and checksum, and block final-ready status if hard violations remain.

9. Proposed normalized PostgreSQL mapping (not frozen)

Dataset/configuration

dataset_versions, institutes, calendar_versions, calendar_days, calendar_periods, calendar_exam_days, calendar_exam_sessions, booking_priority_categories, role_category_caps

Academic master data

departments, rooms, room_features, room_feature_assignments, room_allowed_uses, faculty, faculty_unavailability, faculty_preferences, student_groups, courses

Scheduling

teaching_sessions, session_groups, session_allowed_room_types, timetable_versions, timetable_assignments, solver_runs, solver_run_metrics, validation_runs, validation_violations

Venue booking

booking_requests, booking_windows, booking_allowed_room_types, booking_required_features, booking_mandatory_groups, booking_cancellations, booking_decisions, booking_allocations, approval_tasks, approval_events

Examinations

exams, exam_courses, exam_groups, exam_schedule_versions, exam_assignments, exam_room_allocations

Disruptions/change control

disruption_sets, disruption_changes, change_proposals, change_proposal_items, change_validation_runs, change_approvals, applied_change_sets

Cross-cutting

users, role_assignments, audit_logs, notifications, ai_conversations, ai_messages, ai_tool_executions, import_jobs, export_jobs

Campus Navigator

campuses, buildings, floors, map_assets, map_versions, spaces, doors, entrances, navigation_nodes, navigation_edges, map_processing_jobs, map_review_items, map_publications

Navigator room references should reuse canonical room identity where the same physical room is represented. Do not create a second independent room identity for CR-101.

10. Relationship and integrity principles

• One dataset version contains its source entities and configuration.

• A course has many sessions; each session belongs to one course and one faculty member.

• A session has many student groups through a join table.

• A timetable version has many assignments; each assignment references a session, room, day and start period.

• A booking request has one or more windows, room types, required features and mandatory groups.

• A booking request may have cancellation and decision history; one effective decision is emitted per export version.

• An exam covers many courses and groups and may allocate multiple rooms.

• A disruption set contains ordered changes and creates a derived dataset version only after approval.

• Map versions are immutable; publication points to an approved version.

11. Backend stack and build order

Proposed stack: React + TypeScript + Vite; Tailwind + shadcn/ui; Python + FastAPI; PostgreSQL + PostGIS; SQLAlchemy + Alembic; OR-Tools CP-SAT; Gemini structured outputs/function calling; Pytest plus official validator. Use a modular monolith rather than microservices for the hackathon MVP. Final stack is subject to confirming team Python/TypeScript capability.

• Source audit and requirements freeze

• Data dictionary and ERD

• Database schema and migrations

• Import pipeline and round-trip export tests

• Backend auth, permissions and API contracts

• Timetable solver and validator integration

• Booking and approval workflows

• Exams

• Disruptions and versioning

• Frontend vertical slices against stable API contracts

• Campus Navigator

• Gemini assistant

• End-to-end tests, exports and deployment

Use vertical slices: schema → API → UI → tests. Mock data may support early visual design, but integrated features must use real APIs.

12. AI authority boundaries

• Gemini can parse natural-language requests into editable drafts, ask follow-up questions, explain options and propose changes.

• Gemini cannot write directly to PostgreSQL, approve its own proposals, bypass permissions, override hard constraints or publish without authorised approval.

• Every AI action is schema-validated, permission-checked, domain-validated and audited. Backend code remains authoritative.

13. Discrepancies and open decisions

• Validator CAT_WEIGHT values are hardcoded in validator.py; priority_order and role_category_cap are read from JSON. Preserve both behaviours exactly for competition scoring.

• README says seminar halls and OAT never host regular classes, while timetable validator checks room_types and does not separately enforce this sentence. Inspect source room/session combinations and confirm intended rule before adding a new hard constraint.

• faculty.needs_accessible and student_groups.needs_accessible exist, but H10 checks session.needs_accessible. Any derivation must be an explicit application policy.

• Validator H12 fallback is 4 if faculty max_hours_per_day is absent. Import should flag missing values rather than silently hide them.

• series_id exists but recurrence semantics are not defined by validator.

• requester and cancellation by are display labels, not authenticated user IDs; identity reconciliation is an application workflow.

• department is nullable in booking requests; department authority needs policy.

• Booking validator supports one room per booking allocation; multi-room bookings are not part of current export contract.

• Exam duration_hours is not explicitly checked against AM/PM slot duration.

• Validator does not enforce reviewer roles, no-self-approval or approval chains.

• Public dataset is fictional; do not treat records as live identity data.

• No generated timetable has yet been run through the validator in this audit.

14. Milestone status and acceptance

• Complete: starter ZIP inventory inspected.

• Complete: canonical JSON root keys and source counts inspected.

• Complete: nested source fields documented.

• Complete: README import/export contract reviewed.

• Complete: validator timetable, booking, exam and disruption checks reviewed.

• Pending: approve data dictionary.

• Pending: ER diagram and relational schema review.

• Pending: implement import/export round-trip tests.

• Pending: produce schedule and prove it passes official validator.

Next deliverable: Milestone 2A — ER diagram and relational schema design. This will specify exact tables, column types, keys, relationships, join tables, versioning, audit, constraints, indexes, migration order and source-to-table mappings. Do not generate migrations or application code until the design is reviewed.